

Learning Graph Invariants with PQC

Jules DUPONT

May 19, 2026

Abstract

This document is a report for my project on the use of parametrised quantum circuits (PQCs) for learning a graph invariant – connectivity – realized in the context of the Applied Quantum Algorithms course (QML mini-projects, Topic 2). It is structured as follows: in [section 1](#), the three questions from the theory part are addressed. The implementation of the programming problems is reported in [section 2](#), and in [section 3](#) we present the obtained numerical results and discuss them. All notebooks and experimental data can be found in the [associated Github repository](#).

1 Theory questions

This first section addresses the three theory questions from the project instructions.

1. For any $k \in \mathbb{N}^*$, we have:

$$\begin{aligned}
 h_k(M) &= \langle 0 | U_k | 0 \rangle \\
 &= \langle 0 | \Pi_{i=1}^k (W_i M) | 0 \rangle \\
 &= \langle 0 | W_k M \cdots W_1 M | 0 \rangle \\
 &= \langle 0 | W_k \mathbb{I} M \mathbb{I} \cdots \mathbb{I} W_1 \mathbb{I} M | 0 \rangle \\
 &= \sum_{j_1=0}^{N-1} \cdots \sum_{j_{2k-1}=0}^{N-1} \langle 0 | W_k | j_{2k-1} \rangle \langle j_{2k-1} | M | j_{2k-2} \rangle \cdots \langle j_2 | W_1 | j_1 \rangle \langle j_1 | M | 0 \rangle \\
 &= \sum_{j_1=0}^{N-1} \cdots \sum_{j_{2k-1}=0}^{N-1} w_{(0, j_{2k-1})}^{(k)} m_{(j_{2k-1}, j_{2k-2})} \cdots w_{(j_2, j_1)}^{(1)} m_{(j_1, 0)} \\
 \text{i.e. } h_k(M) &= \sum_{j_1=0}^{N-1} \cdots \sum_{j_{2k-1}=0}^{N-1} m_{(j_{2k-1}, j_{2k-2})} \times \cdots \times m_{(j_1, 0)} \times w_{(j_2, j_1)}^{(1)} \times \cdots \times w_{(0, j_{2k-1})}^{(k)}
 \end{aligned}$$

If we flatten the index pairs into multi-indices with the mapping $m_{i_l} := m_{(j_{2k-2l+1}, j_{2k-2l})}$ for $l = 1, \dots, k-1$, and $m_{i_k} := m_{(j_1, 0)}$ if $i_k < N$, 0 otherwise, and denote $w_{i_1}^{(1)} \cdots w_{i_k}^{(k)}$ the coefficient that ultimately appears in front of the monomial $m_{i_1} \times \cdots \times m_{i_k}$, we obtain:

$$h_k(M) = \sum_{i_1=0}^{n-1} \cdots \sum_{i_k=0}^{n-1} m_{i_1} \times \cdots \times m_{i_k} \times w_{i_1}^{(1)} \cdots w_{i_k}^{(k)}$$

which is a k -th degree multivariate polynomial of the entries of the matrix M .

2. The formula involves k many parameters sets, the $\mathbf{w}^{(j)}$, each of them containing n entries. Therefore, there are exactly $n \times k$ parameters to set.

Now, expanding the sums and using the commutativity of the multiplication we realize that many of the $m_{i_1} \times \cdots \times m_{i_k}$ terms are equal and can be grouped together. For example, with $k = 2$, this is the case with $m_0 \times m_1$ and $m_1 \times m_0$. Still, we have to determine a certain amount of coefficient products, precisely corresponding to the quantity of equations we are looking for. To explicit it, we shall count the number of effectively distinct words of the form $m_{i_1} \times \cdots \times m_{i_k}$, where the $i_j \in \{0, \dots, n-1\}$. This can be viewed as a well-known combinatorial problem in

which one has to place k balls in n boxes (each box may contain zero, one, or more than one ball). This is strictly equivalent to forming a word of length $k + (n - 1)$ out of k zeros interspersed with $n - 1$ ones; one only needs to chose where to place the zeros and hence has $\binom{n+k-1}{k}$ possibilities. (Put differently, we have just recounted the number of multisets of size k from n elements.)

All in all, there are $n \times k$ many parameters to set the coefficients $w_{i_l}^{(j)}$ while having $\binom{n+k-1}{k}$ many equations. We see numerically that, for large n ,

$$\binom{n+k-1}{k} \gg nk$$

meaning that we have much more equations to satisfy than we have parameters to play with. The model is emphatically not maximally expressive: it can only represent a strict subset of all k -th degree multivariate polynomials.

3. The following definition is inspired by [MMC21] and [RBN⁺23].

Definition 1 (Permutation equivariance). Let $U : \{0,1\}^{n \times n} \mapsto U(N)$ be a circuit ansatz parametrized by an adjacency matrix, and let A be an arbitrary adjacency matrix.

We say that U is *permutation equivariant* if for all permutation matrices $P \in \mathcal{S}_{n \times n}$ and corresponding qubit permutation unitaries $\tilde{P}$:

$$U(PAP^T) = \tilde{P}U(A)\tilde{P}^\dagger$$

In other words, an ansatz that is defined by a graph is permutation equivariant if a permutation on the vertices of the graph results in the same ansatz to which we have correspondingly permuted the qubits: relabeling graph nodes and relabeling qubits are the same operation.

This property is highly connected to graph symmetries because it essentially abstracts them away. When two graphs $\mathcal{G}$ and $\mathcal{G}'$ – represented by the adjacency matrices A and A' – are isomorphic, there exists a permutation matrix P such that

$$A' = PAP^T$$

Then, if we decide to use the permutation equivariant parametrized circuit ansatz U , the graph $\mathcal{G}'$ will be encoded by

$$U(A') = U(PAP^T) = \tilde{P}U(A)\tilde{P}^\dagger$$

that is, by the same circuit that encodes $\mathcal{G}$ up to a permutation in the qubits.

Now, if the output of ones PQC is simply defined as the probability of observing the vacuum state $|0^{\otimes n}\rangle$ given that same state as input, one obtains for $\mathcal{G}'$

$$\langle 0^{\otimes n} | U(A') | 0^{\otimes n} \rangle = \langle 0^{\otimes n} | \tilde{P}U(A)\tilde{P}^\dagger | 0^{\otimes n} \rangle = \left(\langle 0^{\otimes n} | \tilde{P} \right) U(A) \left(\tilde{P}^\dagger | 0^{\otimes n} \rangle \right) = \langle 0^{\otimes n} | U(A) | 0^{\otimes n} \rangle$$

that is, the exact same output as for $\mathcal{G}$ and, consequently, the same gradients and optimization landscape for one's circuit parameters.

2 Implementation

In this section, we report the implementation of the data generation step, the quantum model and the classical model.

2.1 Dataset

The data for our (quantum) machine learning problem was first generated following the data generation step described in the project instructions. We generated N random graphs using the Erdős–Rényi model, which builds graphs by choosing each of the possible edges with probability p . Using `networkx`'s `is_connected(G)` function, connected graphs (represented via their adjacency matrices) were given the label 0, and disconnected ones the label 1. As we aimed to have a roughly equal number of connected and disconnected graphs, we varied the probability p according to a uniform distribution roughly centered around the asymptotic threshold of $\frac{\ln n}{n}$ – where n is the number of nodes.

Additionally, as is standard in machine learning, this dataset was divided into training and test sets according to a chosen ratio r .

Data filtering

When choosing the total dataset size N , one must bear in mind the highly symmetrical nature of the problem: the property we are trying to learn – connectivity – is invariant under node permutations, and the circuit ansätze we use for the quantum model are equivariant under node permutations (see below). As such, it is redundant to include isomorphic graphs – graphs that are mathematically equivalent up to a permutation of their nodes – in the training set. Therefore, the training set should strictly consist of non-isomorphic graphs.

For reasonable values of the number of nodes n , these non-isomorphic graphs can be enumerated and counted; their count follows the sequence [OEIS A000088](#). For example, for $n = 5$, there are only 34 unique non-isomorphic graphs, meaning the training set can simply consist of these 34 instances. However, this count grows extremely fast: for $n = 7$, there are already 1044 unique configurations. It rapidly becomes computationally unreasonable to consider all non-isomorphic graphs.

Consequently, we implemented a data filtering procedure. It takes as input a raw training set of size N_{train} , removes all repetitions of isomorphic graphs, and outputs a filtered, strictly non-isomorphic training set of size N'_{train} , as illustrated in [Figure 1](#).

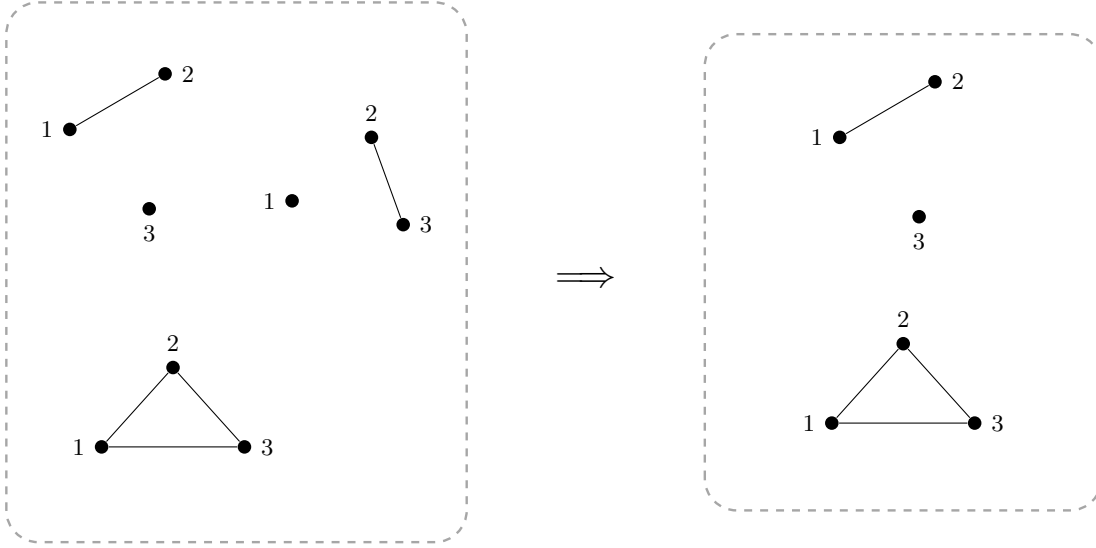


Figure 1: Data filtering procedure on a set of graphs of size 3. The top-rightmost graph from the initial set is isomorphic to the top-leftmost one, hence it is removed. Here $N_{\text{train}} = 3$ and $N'_{\text{train}} = 2$.

In all cases, the test set was made of N_{test} completely random graphs, where N_{test} was chosen to be large enough to provide robust test metrics. The isomorphism data filtering procedure described above was never applied to the test sets.

2.2 Quantum model

The quantum model consists of a standard Variational Quantum Algorithm (VQA) scheme: a Parametrized Quantum Circuit (PQC) is trained to learn the connectivity problem by minimizing a cost function with the help of a classical optimizer.

Parametrized Quantum Circuit

The PQC was defined as suggested in the instructions as a sequence of L layers of the form $W_i M$, where W_i is a variational part and M is a data encoding circuit block, and it was implemented using the open-source Python library PennyLane.

For the variational part, we first implemented the ansatz

$$W_i(\theta_i) = \bigotimes_{\ell=1}^n R_x(\theta_i) \quad (1)$$

which consists of only one trainable parameter per layer (independently of the number of qubits n). The data encoding circuit block M depends by definition on the encoded graph G , so we will subscript M with the corresponding adjacency matrix A , i.e. write M_A .

Now we show that the full circuit ansatz made of this simple variational ansatz $W_i(\theta_i)$ and that data encoding circuit block M_A is permutation equivariant in the sense of Definition 1.

Claim 1. *The circuit ansatz $U : A \rightarrow \Pi_{i=1}^L(W_i(\theta_i)M_A)$ is permutation equivariant.*

Proof. For $P \in \mathcal{S}_{n \times n}$ an arbitrary permutation matrix, we have:

$$M_{PAP^T} = e^{i\tilde{H}_G} = e^{i\tilde{P}H_G\tilde{P}^\dagger} = \tilde{P}e^{iH_G}\tilde{P}^\dagger = \tilde{P}M_A\tilde{P}^\dagger$$

For the third equality, we used the fact that $\tilde{P}$ is unitary, i.e. $\tilde{P}^\dagger\tilde{P} = I$. Then, the result follows by expanding the exponential as a Taylor series, absorbing identities, and reconstructing an exponential. Additionally, and because the parameter θ_i is the same across all qubits/nodes for the layer i , we know that a permutation has no effect on $W_i(\theta_i)$, i.e. $\tilde{P}W_i(\theta_i)\tilde{P}^\dagger = W_i(\theta_i)$. Hence we have:

$$\begin{aligned} U(PAP^T) &= \Pi_{i=1}^L(W_i(\theta_i)M_{PAP^T}) \\ &= \Pi_{i=1}^L\left((\tilde{P}W_i(\theta_i)\tilde{P}^\dagger)(\tilde{P}M_A\tilde{P}^\dagger)\right) \\ &= \Pi_{i=1}^L\tilde{P}(W_i(\theta_i)M_A)\tilde{P}^\dagger \\ &= \tilde{P} \cdot \Pi_{i=1}^L(W_i(\theta_i)M_A) \cdot \tilde{P}^\dagger \\ \text{i.e. } U(PAP^T) &= \tilde{P}U(A)\tilde{P}^\dagger \end{aligned}$$

This shows that U is permutation equivariant. $\square$

With other variational ansätze such as

$$W_i'(\theta_i^{(1)}, \theta_i^{(2)}) = \bigotimes_{\ell=1}^n R_x(\theta_i^{(1)})R_y(\theta_i^{(2)}) \quad (2)$$

$$W_i''(\theta_i^{(1)}, \theta_i^{(2)}, \theta_i^{(3)}) = \bigotimes_{\ell=1}^n R_z(\theta_i^{(3)})R_y(\theta_i^{(2)})R_x(\theta_i^{(1)}) \quad (3)$$

which were both implemented in the notebook and used in our experiments, or any other definition of the variational layers that is not impacted by a permutation on the qubits $\tilde{P}$, the proof works exactly the same.

As a follow up, we allowed parametrization in the data encoding layers, that is rather use

$$M_{j,A} = e^{i\gamma_j H_G}$$

where γ is a vector of trainable parameters. Since we still have $M_{j,PAP^T} = \tilde{P}M_{j,A}\tilde{P}^\dagger$, the circuit ansatz remains permutation equivariant.

The output of this PQC was set to be the probability of observing the vacuum state $|0\rangle^{\otimes n}$, that is $\langle 0|U_L|0\rangle$ – corresponding to the k -th degree multivariate polynomial $h_L(M)$ studied in the theory questions. Ultimately, after tuning the parameters, this probability would output a score predicting the likelihood of the graph encoded in the PQC being disconnected: after discretization around the threshold 0.5, a value of 0 would suggest a connected graph, 1 for a disconnected graph.

For the input quantum state, the first unitary of the PQC is the data encoding circuit block M_A , which is essentially a product of Ising-ZZ gates. Because they only add a global phase to $|0\rangle^{\otimes n}$, the input quantum state should rather be a superposition, for instance $|+\rangle^{\otimes n}$ which can be obtained by applying a layer of Hadamard gates – and the last can be proven to commute with any $\tilde{P}$.

Classical optimization loop

Regarding the classical optimization loop, we defined our cost function as the mean over the Binary Cross-Entropy (BCE) losses between the PQC continuous prediction and the ground truth label over the entire training set. Then, we used a classical Adam optimizer to correspondingly vary the parameters.

This mean BCE loss was a key metric when assessing the training of a model, although another metric that was logged is the accuracy – defined as the proportion of correct predictions after discretization. Both these metrics were analyzed over the training set and the test set in order to evaluate the generalization capacity of the model. Each training was also evaluated by visually inspecting a confusion matrix, and in order to assess the convergence of the training of our models we visually inspected the loss curves.

2.3 Classical model

For the classical model, we made use of the `sklearn` library. Its built-in polynomial model `PolynomialFeatures` was pipelined with a `LogisticRegression`, applying the free weights $\mathbf{w}$ and appending a sigmoid for binary classification.

At first and as it seems to be what the project instructions suggest, we tried to train this classical model on the matrices $M = e^{iH_G}$, where H_G is the Ising Hamiltonian. But since H_G acts on an n -qubit Hilbert space of dimension 2^n , it has $(2^n)^2 = 2^{2n}$ entries. So does M , and because its entries are also complex numbers, this actually means storing $2 \times 2^{2n} = 2^{2n+1}$ real features. Then, the total number of terms generated by `PolynomialFeatures` and, consequently, the number of free parameters for the `LogisticRegression` to handle is $\sum_{\ell=1}^k \binom{2^{2n+1} + \ell - 1}{\ell}$. As of $k = 2$ and $n = 5$, this represents already 2,100,224 free parameters and rapidly this becomes untractable.

As a consequence we decided to train the classical model directly on the adjacency matrices A_i . They are represented by only n^2 features, so this is far more tractable: even for $k = 2$ and $n = 10$, that represents only 5,150 free parameters.

As for the quantum model, a key metric that was used is the accuracy, evaluated on both the training and test sets.

The training of all models – quantum and classical – was logged under a separate `.jsonl` file, allowing for tracking of the evolution of key metrics such as the accuracies and making it easier to draw the plots presented in the next section.

3 Results & Discussions

In this section we present some of the results that we obtained while training the quantum and classical models, and we try to answer the questions raised in the project instructions.

3.1 First experiment

In this first experiment we tried to learn the connectivity problem on graphs of size $n = 8$ with our quantum model. We fixed the number of layers to $L = 2$ but ran the experiment with the first and the second variational ansätze, as defined in Equation 1 and Equation 2. Regarding the used dataset, we generated $N = 500$ random graphs – 247 of which were connected – divided into a training and a test sets following a ratio $r = 0.2$, without any of the filtering mentioned earlier. We let the training run for $E = 50$ epochs, allowed the parametrization of the data encoding layers (parameter γ) and used an initial learning rate for the optimizer of $l = 0.1$.

Figure 2a shows the evolution of the cost function value (mean BCE loss) and the train and test accuracies over the training of the quantum model (first ansatz).

Although the cost function globally decreases throughout the training, we see that the model completely fails at learning the connectivity problem as the accuracy on both the training and test sets remains constant around 50% (and both sets contain roughly as many connected and disconnected graphs).

This is confirmed by the inspection of the confusion matrix represented in Figure 2b: all graphs are classified as connected.

Figure 3a shows the same curves, now using the second variational ansatz.

The quantum model is now better at learning the connectivity problem: an accuracy of $\sim 73\%$ over the test set is reached. We can already remark the oscillations in the train and test accuracies, a phenomenon that will be discussed later.

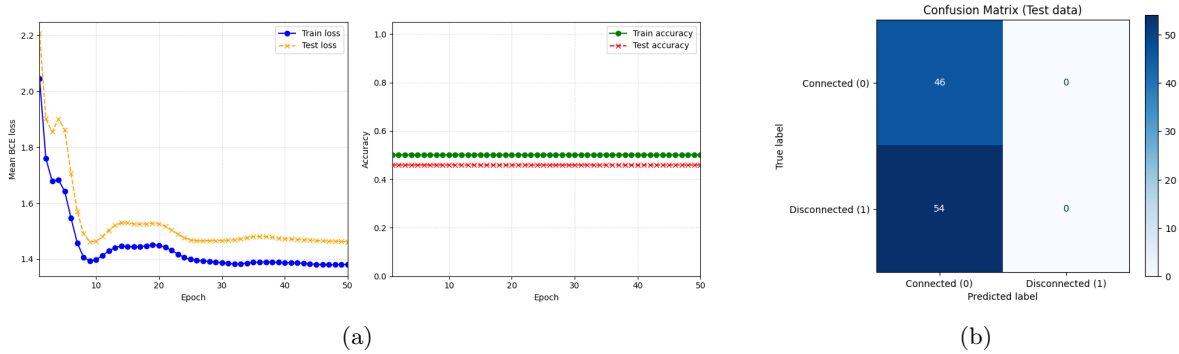


Figure 2: (a) Evolution of the mean BCE loss and accuracies throughout training, and (b) confusion matrix on the test data, for the quantum model with the first variational ansatz. See text for hyperparameter values.

The performance improvement is confirmed by the confusion matrix represented in Figure 3b, which is more diagonal than earlier.

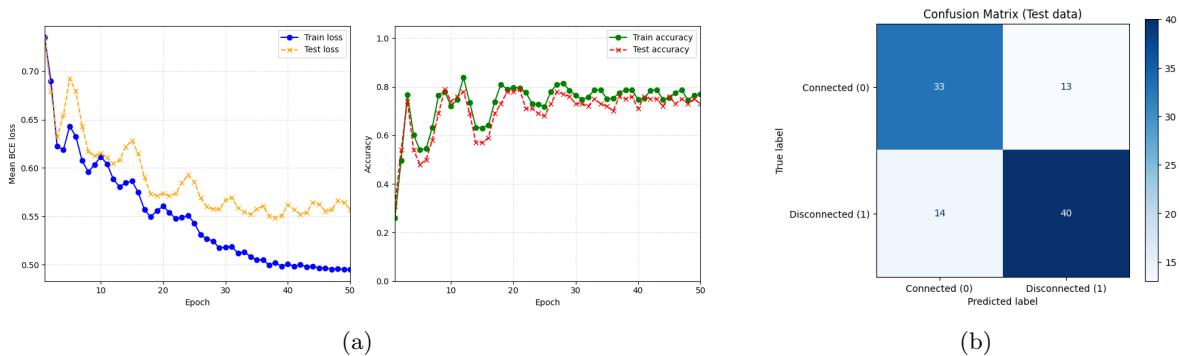


Figure 3: (a) Evolution of the mean BCE loss, and accuracies throughout training, and (b) confusion matrix on the test data, for the quantum model with the second variational ansatz. See text for hyperparameter values.

3.2 Number of reuploading layers and variational ansatz

In this other experiment, which mainly aimed at answering question e- from the project instructions, we trained the quantum model for the two first variational ansätze and for different numbers of reuploading layers (from $L = 1$ to $L = 5$). All metrics were averaged by training the model for each pair (L , variational ansatz) over 5 different random seeds, only 2 for ($L = 4$, 1 param/layer) and ($L = 5$, 1 param/layer).

We again worked with graphs of size $n = 8$, a dataset of initial size $N = 500$ but now applying the aforementioned data filtering procedure on the training set (impact discussed later). We always let the training run for $E = 50$ epochs, allowed the parametrization of the data encoding layers and used an initial learning rate for the optimizer of $l = 0.1$.

In Figure 4, we plot the evolution of the final test accuracy (i.e. the accuracy over the test set, evaluated after the training) against the number of re-uploading layers for both variational ansätze.

We see that with the first variational ansatz, the quantum model consistently fails to learn the connectivity problem (final test accuracies $\sim 50\%$).¹ With the second variational ansatz though, the quantum model achieves reasonable classification performances. For $L = 1$, this is not very clear and the performance is rather widespread across the runs. But as of $L = 2$, the reached final test accuracy

¹Although we want to note that during the experiments, in other contexts, we came across sets of hyperparameters where the model with the first variational ansatz did manage to slightly learn the connectivity problem.

is relatively high and stable ($75.8 \pm 2.2\%$). As we continue increasing L , the final test accuracy does not continue increasing though.

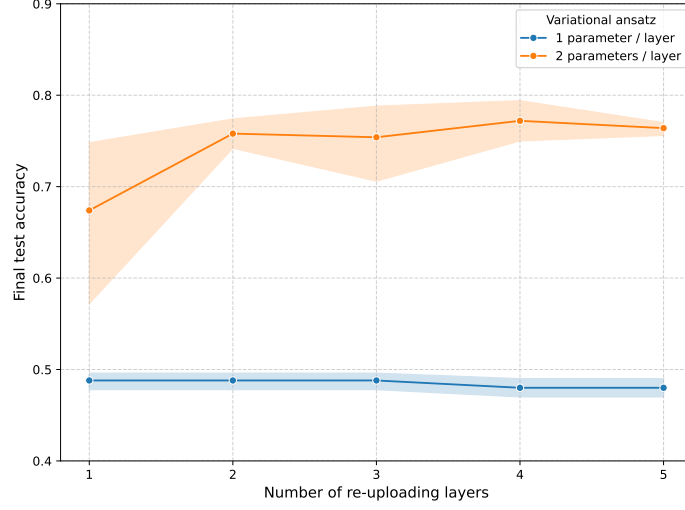


Figure 4: Final test accuracy against the number of re-uploading layers L . See text for hyperparameter values.

However, Figure 5 clearly shows how the resource consumption (expressed through the average time per epoch) of the training scales more rapidly with L when the second variational layer is used. Indeed, the circuit ansatz that uses the second variational layer requires the training of L more parameters than the one that uses the first variational layer.

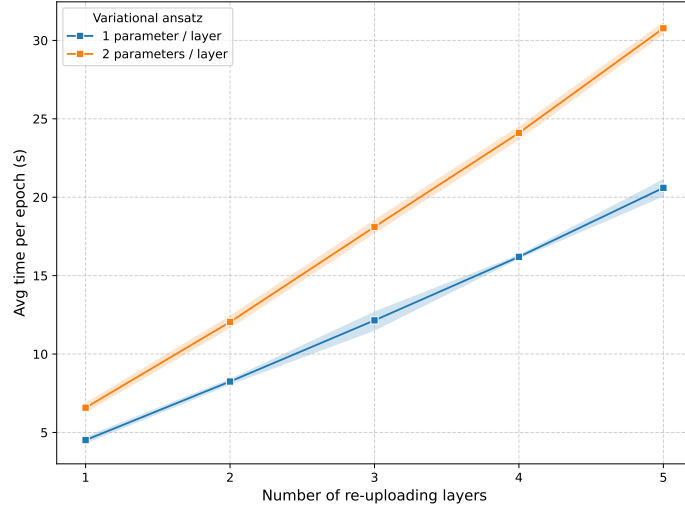


Figure 5: Average time per epoch of the training (in seconds) against the number of re-uploading layers L . See text for hyperparameter values.

3.3 Data filtering

We conducted another experiment to evaluate the significance of the data filtering procedure presented earlier. Still with $n = 8$, we trained the quantum model (with $L = 2$ layers, and the second variational ansatz) until it reached a test accuracy $> 75\%$ or 50 epochs, for both the cases were the training set was filtered and not filtered.

Table 1 summarizes the results of this experiment, repeated over 10 runs.

When the training set is filtered, the training epochs end faster. This was expectable, since the model has then less data points to train on ($\sim -30\%$). Yet, the mean wall time to reach the threshold ($> 75\%$ accuracy) does not benefit the same shrinkage on average. Indeed, interestingly the model takes on average more training epochs to reach threshold when the training set is filtered than when it is not. This indicates that although our circuit ansatz is equivariant and should understand that the connectivity problem is invariant under nodes permutation, augmenting the training set with isomorphic graphs does still have a positive impact on the training.

Table 1: Evaluation of the impact of the data filtering procedure on training performance metrics

Metric	Filtered	Not filtered
Success rate ^a	8/10	9/10
Mean time per epoch ^b	21.5 ± 0.5s	29.3 ± 1.0 s
Mean wall time to threshold ^b	346.2 ± 153.3s	377.9 ± 260.0 s
Mean epoch reached threshold ^b	16.0 ± 6.9	12.8 ± 8.4

^a 'Success' is defined as reaching $\geq 75\%$ test accuracy within 50 epochs.

^b Failed runs (where the model did not reach $> 75\%$ test accuracy within 50 epochs) are excluded from the time and epoch means.

3.4 Reuploading layers parametrization

In all previous experiments, we allowed parametrization in the data encoding layers through an additional trainable parameter vector γ as it appeared crucial for the model to successfully learn the connectivity problem from first experiments.

In order to be more rigorous, we conducted a paired sample t-test to statistically prove the significance of the data encoding layers parametrization.

For that, we defined the two following hypotheses:

- Null hypothesis ($\mathcal{H}_0$): Parameterizing the data encoding layers has no significant effect on the model's performance. The true mean difference in final test accuracy between the parameterized model and the fixed model is zero.
- Alternative hypothesis ($\mathcal{H}_A$): Parameterizing the data encoding block significantly improves the model's performance. The true mean difference is greater than zero.

From our observation, $\mathcal{H}_A$ should be true. In order to accept or reject $\mathcal{H}_0$, we conducted the following paired experiment: for each trial i from 1 to $N = 10$,

1. Pick a random seed s_i .
2. Train a model with hyperparameters $n = 7$, $L = 2$, second ansatz, $N = 500$, $r = 0.2$, $l = 0.1$ and with the implementation which allows parametrization in the data encoding layers, manually setting the random seed to s_i . Record the final test accuracy, call it X_i .
3. Train a model with the same hyperparameters but now with the implementation which does not allow parametrization in the data encoding layers, and again manually setting the random seed to s_i . In particular, the dataset is expected to be the exact same and we stop after the same number of epochs. Record the final test accuracy, call it Y_i .
4. Record the paired difference $d_i = Y_i - X_i$

With the data obtained throughout these 10 trials and using the `scipy.stats` library, we obtained a t -statistic of 6.4 and a p -value of $p = 0.0001$ for rejecting the null hypothesis $\mathcal{H}_0$. This means that we can say with high confidence that parametrizing the data encoding layers significantly improves model performance and this confirms our earlier observation.²

²In fact the unparametrized model barely ever learned the connectivity problem, as $\mu_{\text{unparam}} = 0.53 \pm 0.04$.

3.5 Oscillations

It is interesting to observe how not only the losses but also the accuracies evolve by following oscillations, as in Figure 6. This is likely due to the periodicity of the $\theta \mapsto R_n(\theta)$ parametrized unitaries, but complicates the question of choosing when to stop the training. We could have implemented an exponential learning rate decay, as was done in [MMC21], in order to avoid these oscillations.

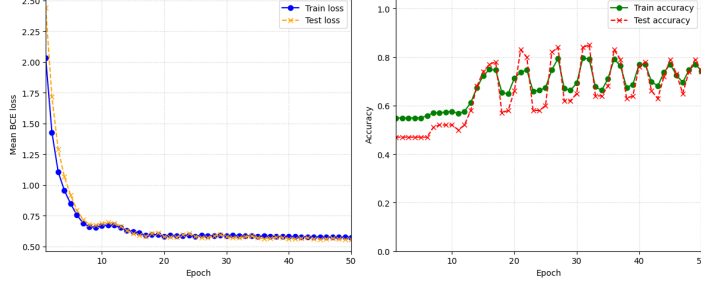


Figure 6: Fledged oscillations in the training and test accuracies during the training of a quantum model. This happens after ~ 15 epochs in spite of the losses appearing to converge.

3.6 Performance on unseen graph isomorphisms

The fact that the train and test accuracies always stayed relatively close during our trainings indicates that there is likely no overfitting. But how do the models perform on graph isomorphisms they have certainly not encountered during their training? This should tell more about the generalization capacity of the models.

To answer this question, we implemented a `generate_novel_test_set(train_data, num_nodes, num_new_graphs)` utility that generates `num_new_graphs` graphs that are isomorphic to no graph from the `train_data` set. Then, we evaluated our models on these novel test sets by computing an accuracy.

The experiment whose results are summarized in Table 2 was conducted on graphs of size $n = 7$, still with a dataset of initial size $N = 500$ and split following a ratio $r = 0.2$. The training set was filtered according to the aforementioned procedure, and all metrics were averaged over 10 runs.

The accuracy obtained on the generated novel test set (`num_new_graphs` = 100 non-isomorphic graphs sharing an isomorphism with no graph from the training set) is comparable to the accuracy obtained on the standard test set, which is reasonable.³ This says the PQC is not just memorising the graph isomorphisms it is trained on: it actually encodes graph structure properties.

Metric	Quantum models
Train accuracy	$75.2 \pm 3.6\%$
Test accuracy	$74.4 \pm 9.2\%$
Novel test accuracy	$80.2 \pm 4.1\%$
Total training time	$262 \pm 9\text{s}$

Table 2: Performance of quantum models trained on 7-node graphs. Aggregated results over 10 runs. Uncertainties represent one standard deviation across seeds.

3.7 Comparison with the classical model

As was discussed earlier, the chosen baseline for the classical model consisted of learning the adjacency matrices directly – and not the encoded matrix.

One experiment consisted of comparing the learning of the connectivity problem for the quantum model in the settings of the last experiment to that of the classical mode, with the exact same dataset ($n = 7$, $N = 500$, $r = 0.2$, data filtering) and again averaging over 10 runs.

³It is even slightly higher, but this is not statistically significant.

Table 3 shows the results of this experiment. Although the training of the classical model is almost instantaneous compared to the training of the quantum model, the classical model appears to suffer from overfitting: it learns perfectly on the graphs from the training set, but the achieved test accuracy hardly beats the quantum model, indicating that it struggles to generalize. This is confirmed by observing the rather low accuracy on the aforementioned novel test set.

Metric	Quantum models	Classical models
Train accuracy	$75.2 \pm 3.6\%$	$100.0 \pm 0.1\%$
Test accuracy	$74.4 \pm 9.2\%$	$76.0 \pm 2.9\%$
Novel test accuracy	$80.2 \pm 4.1\%$	$78.9 \pm 3.5\%$
Total training time	$262 \pm 9\text{s}$	$< 1\text{s}$

Table 3: Comparison of quantum and classical models, averaged over 10 runs. Uncertainties represent one standard deviation across seeds.

3.8 Towards better performance

Across the previous experiments we observed settings where the final train and test accuracies of quantum models more or less converged around 75-80% – with the oscillations mentioned earlier – but we never observed much better performances.

As suggested by the project instructions and with the aim to surpass this plateau, we came up with the following other equivariant variational ansatz, with more trainable parameters in each layer:

$$W_i(\theta_i^{(1)}, \theta_i^{(2)}, \theta_i^{(3)}, \phi_i) = \left(\prod_{\ell < \ell'} e^{i\phi_i X_\ell X_{\ell'}} \right) \left(\bigotimes_{\ell=0}^{n-1} R_z(\theta_i^{(3)}) R_y(\theta_i^{(2)}) R_x(\theta_i^{(1)}) \right) \quad (4)$$

Figure 7 shows a circuit diagram for this variational layer, with $n_{\text{num_nodes}} = 4$.

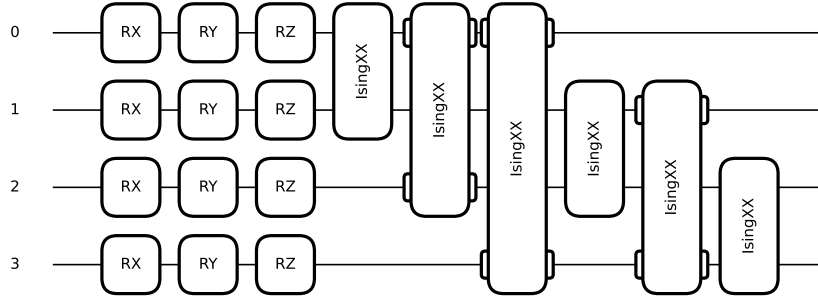


Figure 7: Circuit diagram for our improved ansatz, with $n = 4$.

The third rotation angle $\theta_i^{(3)}$ completes the Euler decomposition, making the right part universal for single-qubit unitary operations. As before, note that the $\theta_i^{(j)}$ are qubit-independent; this is crucial for preserving the equivariance property, as will be discussed later. As for the left part, we chose to add all-to-all IsingXX connectivity. They are also parametrized by a shared angle ϕ_i , and their role is to generate entanglement from the variational layer. We chose IsingXX over IsingZZ, for the latter are already present in the M_A layer.

Claim 2. *The circuit ansatz U made of the variational layer from Equation 4 is permutation equivariant.*

Proof. From the proof of Claim 1 it is clear that the single-qubit rotations part of the variational layer is permutation equivariant, and we only need to show that the new entangling part remains unchanged

by a permutation on the qubits $\tilde{P}$. And we have:

$$\begin{aligned}
\tilde{P} \left(\prod_{\ell < \ell'} e^{i\phi_i X_\ell X_{\ell'}} \right) \tilde{P}^\dagger &= \prod_{\ell < \ell'} \tilde{P} e^{i\phi_i X_\ell X_{\ell'}} \tilde{P}^\dagger \\
&= \prod_{\ell < \ell'} e^{i\phi_i \tilde{P} X_\ell X_{\ell'} \tilde{P}^\dagger} \\
&= \prod_{\ell < \ell'} e^{i\phi_i X_{\pi(\ell)} X_{\pi(\ell')}} \\
&= \prod_{\ell < \ell'} e^{i\phi_i X_\ell X_{\ell'}}
\end{aligned}$$

where π is the corresponding permutation acting on indices, and the last step being just a reordering of the product terms. This shows what we wanted to show. $\square$

In Table 4 we show the averaged performance of quantum models equipped with $L = 2$ layers of this improved ansatz ($N = 500$, $r = 0.2$, $l = 0.1$) against a baseline obtained with $L = 2$ layers of the second variational ansatz.

Metric	Baseline	Improved ansatz
Train accuracy	$73.7 \pm 1.8\%$	$75.9 \pm 2.2\%$
Test accuracy	$75.8 \pm 1.9\%$	$78.0 \pm 3.6\%$
Novel test accuracy	$74.4 \pm 3.0\%$	$74.6 \pm 4.2\%$
Total training time	$602 \pm 23\text{s}$	$721 \pm 12\text{s}$

Table 4: Evaluation of the improved ansatz, averaged over 5 runs. Uncertainties represent one standard deviation across seeds.

In this specific experiment, our new ansatz does not significantly outperform the earlier one. Accuracies have slightly increased (especially, the test accuracy), but not dramatically and the spread is quite important. Also, the training takes longer: there are more parameters to optimize.

Also note that adding too many parameters to a PQC often leads to barren plateaus, but according to the authors of [SLN⁺24] this should not be the case when using such permutation equivariant ansätze.

3.9 Non-equivariant ansatz

Does the permutation equivariance property of the circuit ansatz really have an impact in experiments of such scales?

Looking back at the proof of Claim 1, one way to break the permutation equivariance property is to modify the impact of the permutation $\tilde{P}$ on $W_i(\theta_i)$. This can be done by making the θ_i qubit-dependent, i.e. $\theta_i \mapsto \boldsymbol{\theta}_i = (\theta_i^{(1)}, \dots, \theta_i^{(n)})$.

As a last experiment, we then broke the equivariance of the second variational ansatz by making all the rotation angles qubit-dependent. We used $n = 8$, and trained quantum models with $L = 2$ for five of the random seeds used in an earlier experiment. Table 5 summarizes how the averaged performance metrics evolved.

Metric	Equivariant	Non-equivariant
Train accuracy	$73.7 \pm 1.8\%$	$70.0 \pm 2.1\%$
Test accuracy	$75.8 \pm 1.9\%$	$69.6 \pm 2.0\%$
Mean avg time	$12.0 \pm 0.5\text{s}$	$12.6 \pm 0.5\text{s}$

Table 5: Comparison of the equivariant and non-equivariant ansätze, averaged over 5 runs. Uncertainties represent one standard deviation across seeds.

What we can see is that the final test accuracy does decrease when the permutation equivariance property is broken, from an average of 76% to an average of 70%. But the change is not dramatic, and the quantum model still manages to become better than random guessing, in similar times.

A follow-up experiment for this question would then be to evaluate how both models behave when tested on graphs that are isomorphic to graphs present in the training set: we expect the equivariant ansatz to do better than the non-equivariant ansatz.

Conclusion and outlook

In this mini-project, we explored the use of permutation equivariant ansätze for embedding bias when learning a simple graph invariant – connectivity. We saw that quantum models are capable of achieving reasonable performance even though they are trained on small datasets, demonstrating a good capacity of generalization (3.1, 3.2, 3.6, 3.8). Most interestingly, the number of parameters they require scales only linearly with the size of the graphs. However, the theoretical advantages of using permutation equivariant ansätze are not obvious experimentally (3.3, 3.9). On the other hand, addressing the learning problem by fitting a classical model as described in the project instructions requires optimizing a much faster growing number of free parameters. On the scale of our experiments and when learning the adjacency matrices directly this is tractable but the results exhibit slight overfitting (3.7). But the quantum versus classical comparison remains tedious, as the graph invariant we are working on is a toy example and can readily be handled via other classical methods.

While we already started exploring questions on the impact of different design choices on the performance of quantum models (3.3, 3.4, 3.5, 3.6, 3.8, 3.9), a natural follow-up for this project would be to conduct a more exhaustive ablation study.

Also, it would be interesting to extend this work by addressing other of the many graph invariants mentioned in [RBN⁺23, MMC21].

References

- [MMC21] Péter Mernyei, Konstantinos Meichanetzidis, and İsmail İlkan Ceylan. Equivariant quantum graph circuits. *CoRR*, abs/2112.05261, 2021.
- [RBN⁺23] Michael Ragone, Paolo Braccia, Quynh T. Nguyen, Louis Schatzki, Patrick J. Coles, Frédéric Sauvage, Martin Larocca, and M. Cerezo. Representation theory for geometric quantum machine learning, 2023.
- [SLN⁺24] Louis Schatzki, Martín Larocca, Quynh T. Nguyen, Frédéric Sauvage, and M. Cerezo. Theoretical guarantees for permutation-equivariant quantum neural networks. *npj Quantum Information*, 10(1), January 2024.